msieve on NVIDIA GPUs

Polynomial Selection, Lattice Sieving, Linear Algebra and Postprocessing in a GPU Fork of msieve

Stefan Teleman <stefan.teleman@gmail.com>
in collaboration with Claude Opus 5.5 (Anthropic)

November 10, 2026

Abstract

msieve's number field sieve begins with polynomial selection: a collision search for candidate polynomials (stage 1), size optimization of each candidate, and root optimization of the best (stage 2). msieve has run stage 1 on NVIDIA GPUs for years; on a current card the shipped code kept the GPU 97% idle and silently discarded every collision after the 999th of each batch. This document describes a fork of msieve SVN trunk r1113 that fixes the lost collisions, builds for sm_120 (Blackwell), moves the host-side root arithmetic to 64-bit Montgomery code and the GPU, overlaps special-q generation with the GPU, runs stage 2 on every CPU, and finally moves size optimization itself to the GPU, with its multiple-precision integers in NVIDIA CGBN and its floating-point minimization reproducing the CPU results bit for bit. Root optimization, the main cost above about 150 digits, is reorganized as a batched root sieve whose minimizations and line sieves run on the GPU in batches of hundreds per polynomial. On one leading coefficient of a 120-digit number, stage 1 runs in 6.8 s against 778.6 s for the CPU-only build, and size optimization on the GPU is faster than all 32 CPU threads of the host while using one of them; on one 155-digit coefficient, root optimization takes 102.6 s against 195.6 s on 32 CPU threads and keeps every polynomial msieve keeps. Every step was checked by comparing outputs, not timings: hit sets and size-optimized polynomials are identical to the reference code, and the batched root sieve gives identical results on the CPU and the GPU. The block Lanczos linear algebra and msieve's line siever run on the GPU too, the siever with exactly the CPU siever's relations, and a new GPU lattice siever sieves a 95-digit number in 43.7 s against 9.2 hours for the GPU line siever. The whole factorization was then tuned end to end (§11): a 120-digit number from scratch went from 52.7 to 9.8 minutes on one GPU (CADO-NFS on 32 CPU threads: 28.6 minutes), and 130- and 140-digit numbers take 35.4 and 109.6 minutes.

Authorship. This fork was developed by Stefan Teleman in collaboration with Claude Opus 5.5, Anthropic's AI model, working in Claude Code: the code, the measurements, the README and this document were produced together, each change built, checked against the reference outputs and timed on the development machine before it was committed.

Hardware. All measurements were made on an NVIDIA RTX PRO 4500 Blackwell with an Intel i9-13900K, a development machine. The intended production configurations are the NVIDIA RTX PRO 6000 Blackwell (minimum) and the B200 (optimal); nothing in this document was measured on them, and any statement about them is an untested estimate.

msieve is public domain. The files this fork adds (`stage1_sizeopt_gpu.{c,h}`, `sizeopt_core.{cu,h}`, `root_sieve_gpu.{c,h}`, `root_sieve_batch.c`, `line_sieve_core.h`, `murphy_core.h`, `alpha_core.h`, `lanczos_core.cu`, `lanczos_gpu.{c,h}`, `sieve_core.{cu,h}`, `sieve_line_gpu.c`, `lasieve_core.{cu,h}`, `sieve_lattice_gpu.c`, `relpar.{c,h}`, `relstore.{c,h}`, `sqrt_gpu_mul.{c,h}`, `sqrtmul_core.{cu,h}`, the test programs) and this document are under the MIT license, Copyright © 2026 Stefan Teleman.

Contents

1 Overview **3**
1.1 Polynomial selection in msieve . . . 3
1.2 Hardware and test inputs . . . 3
1.3 Summary of results . . . 3

2 The shipped GPU stage 1 **3**
2.1 999 hits per batch . . . 4

3 Stage 1 on the host and the GPU **4**
3.1 64-bit and Montgomery root lifting . . . 4
3.2 Lifting on the GPU . . . 4
3.3 Incremental CRT . . . 4
3.4 Special-q producer threads . . . 5

4 Stage 2 is the real cost **5**
4.1 Threaded stage 2 . . . 5
4.2 A pool freed with work in it . . . 5

5 End to end: which search to keep **5**

6 Size optimization on the GPU **6**
6.1 Data flow . . . 6
6.2 Integers in CGBN . . . 6
6.3 Doubles, bit for bit . . . 6
6.4 Making the minimization fast . . . 7
6.5 Results . . . 8

7 Root optimization on the GPU **8**
7.1 Step 1: msieve's code, calls on the GPU . . . 8
7.2 Throughput is not the problem . . . 8
7.3 The batched root sieve . . . 8
7.4 The line sieve on the GPU . . . 9
7.5 The engine . . . 9
7.6 Results . . . 10
7.7 The last step: root properties and Murphy E . . . 10

8 Linear algebra **13**

9 Line sieving on the GPU **14**
9.1 Results . . . 14

10 Lattice sieving on the GPU **15**
10.1 Three bugs that lost relations . . . 16
10.2 Results . . . 17
10.3 Postprocessing on all CPUs and in memory . . . 18
10.4 RSA-2048: an extrapolated estimate . . . 21

11 The whole run, end to end (November 10) **22**
11.1 Polynomial selection: shorter time limits . . . 23
11.2 The lattice siever: 32-bit arithmetic, a sieve array kept in L2 . . . 23
11.3 1/128 of the time limits . . . 23
11.4 `la_tile`: vector loads, no bank conflicts, smaller tiles . . . 24

11.5 The square root: more dependencies, products on the GPU 24
11.6 Free relations 24
11.7 Larger inputs: c130 and c140 24

12 Build, settings and the C11 code **24**

13 Open points **25**

1 Overview

1.1 Polynomial selection in msieve

For an input N, NFS needs an algebraic polynomial f of degree d (4–6) and a linear polynomial $g = Px - m$ sharing a root modulo N. msieve searches for them in three stages (`gnfs/poly/`):

1. **Stage 1** (`stage1/`): for each leading coefficient a_d, a Kleinjung-style collision search over products $P = p \cdot q$ of a special-q and progressions p, looking for m with f's next coefficient small. Each *hit* is a triple (a_d, P, m). The GPU version sorts progressions on the card and reports collisions in a `found_array`.
2. **Size optimization** (`stage2/stage2.c`, `optimize.c`): each hit is expanded into f (`pol_expand`), then rotated and translated to minimize a norm (`optimize_initial`, built on `minimize()` in `common/minimize.c`); a projective α is added and the result compared with the stage 2 bound. About one hit in 4000 passes.
3. **Root optimization** (`stage2/root_sieve*.c`): a root sieve over further rotations, scored by Murphy's E; the best polynomials are kept.

`-np` runs all three, `-np1`/`-nps`/`-npr` single stages with files in between (`.m`, `.ms`, `.p`).

1.2 Hardware and test inputs

NVIDIA RTX PRO 4500 Blackwell (82 SMs, sm_120, 32 GB), CUDA 13.2, driver 595.71; Intel Core i9-13900K (32 threads). The main input is a 120-digit semiprime (`runs/c120.txt`), searched on one leading coefficient, 100104; a 95-digit (degree 4) and a 140-digit input check the other code paths.

1.3 Summary of results

Measurement (c120, one coefficient)	Before	After
Stage 1, CPU-only build vs this fork (2,588,149 hits)	778.6 s	6.8 s
Stage 1 hits kept by the shipped GPU code	38,961	all
Stage 2 for all hits of the coefficient (1 vs 31 threads)	809.5 s	35.6 s
Size optimization, 32 CPU threads vs GPU (1.78 M collisions)	20.5 s	15.6 s
Root optimization, c155 coefficient, 32 CPU threads vs GPU	195.6 s	102.6 s
Block Lanczos, 600k synthetic matrix, 16 CPU threads vs GPU	116.0 s	15.8 s
Line sieving, c95, b 1–300, CPU vs GPU siever (same relations)	97.8 s	5.8 s
c95 sieving, GPU line siever vs GPU lattice siever	33,210 s	43.7 s
c120 from scratch, `./msieve` (Nov 9 vs Nov 10, §11)	52.7 min	9.8 min
c140 from scratch (CADO-NFS GPU fork: 75.9 min)	—	109.6 min

2 The shipped GPU stage 1

Measured first, on the unmodified code: the CPU-only build took 778.6 s for one coefficient; the GPU build 30.6 s, but it reported only 38,961 hits, exactly 39×999. Building for sm_120 (the Makefile had no Blackwell target; a `stage1_core_sm120.ptx` rule and loader branch were added,

and `cub/` gained `compute_120`) changed nothing: nsys showed the GPU idle 97% of the time. Two problems, then: the hit count, and a host that could not feed the card.

2.1 999 hits per batch

The collision kernels append to `found_array`, whose slot 0 holds the count; `FOUND_ARRAY_SIZE` was 1000, and the kernels stop writing when it is full. Nothing reported the overflow. A batch typically finds tens of thousands of collisions, so 39 batches gave 39×999. The array is now 2^{20} entries; `check_found_array` copies the count first, warns if it exceeds the array, then copies only the used entries (the full array is 24 MB).

With every hit kept, the GPU code still searched only special-q with at least d roots, while the CPU code takes any; `Q_ROOTS_MIN` (default: the degree) makes this a compile-time choice. With the cap fixed the default search finds 61% of the full search's hits (1,580,311 of 2,588,149); which is better is an end-to-end question (§5).

3 Stage 1 on the host and the GPU

perf showed where the host time went once the GPU was waiting: lifting special-q roots from mod q to mod q^2 in GMP, combining roots of composite q by CRT in GMP, and enumerating special-q serially. Each change below kept the hit set identical (sorted, distinct lines of the `.m` file).

Change (`Q_ROOTS_MIN=1`, 2,588,149 hits each)	Wall
cap fixed, full search	65.8 s
64-bit root lifting	36.4 s
Montgomery multiplication in the lifting	30.0 s
special-q roots lifted on the GPU (kernel 26 ms in all)	18.5 s
incremental CRT	12.5 s
4 special-q producer threads	8.3 s
hits echoed to stdout only with `-v`	6.74–6.88 s

3.1 64-bit and Montgomery root lifting

For $p < 2^{32}$ every quantity mod p^2 fits in 64 bits. `lift_roots_64` (`stage1_roots.c`) does the Hensel step with `mulmod64`/`powmod64` and, for odd p with $p^2 < 2^{63}$, in Montgomery form ($R = 2^{64}$): exact division by p is a multiplication by $p^{-1} \bmod 2^{64}$, and the inverses of all roots come from one Montgomery batch inversion. `combine_roots` gained a 128-bit path for $p < 2^{32}$.

3.2 Lifting on the GPU

The special-q roots were lifted on the host before the batch was copied. The kernel `lift_qroots_q32` (in `stage1_core.cu`) does it on the device, one thread per root, with N and m_0 in constant memory and a 64-bit Montgomery reduction (`lift_mont_*`); `sieve_fb_set_defer_lift` tells the host enumeration to skip the lift. The lifted roots are copied back because the host rebuilds each hit from them (until the size optimization moved there too, §6).

3.3 Incremental CRT

A composite special-q has the product of its factors' root counts as roots, and the old code recombined every root from scratch. `get_enum_roots_crt` does Garner's CRT incrementally with a cache per level of the factorization, so consecutive q sharing factors reuse the partial products; it keeps msieve's root order and its `MAX_ROOTS` truncation, so the hits are unchanged.

3.4 Special-q producer threads

`MSIEVE_Q_THREADS` threads (default 4) enumerate special-q, each over its partition (`sieve_fb_clone`: by the first factor modulo the thread count, or the prime itself), and fill a ring of $n + 2$ batch slots (FREE / FILLING / FULL / DRAINING) that the GPU thread drains. The ring is also the double buffering of the host-to-device copies. More than 4 producers measured no faster.

PCIe traffic for the whole coefficient was 87 ms host-to-device and 80 ms device-to-host, 0.9% of the run: the transfers were never the problem.

4 Stage 2 is the real cost

With all hits kept, size optimization of 10,000 sampled hits took 2.89 s on one thread, 0.29 ms each: the 2.6 M hits of one coefficient would need about 750 s, over 100 times stage 1. None of the 10,000 passed the size bound (one in about 4000 does). Speeding up stage 1 further was pointless; two attempts confirmed it: handling hits on several threads inside stage 1 was *slower* (4, 8, 16 threads: 9.1, 12.2, 18.6 s against 6.8 s), because msieve's stage 2 pool had a single thread behind a blocking queue and the hand-off allocates.

4.1 Threaded stage 2

`poly_skew.c` now builds `MSIEVE_STAGE2_THREADS` workers (default: the online CPUs), each with its own `poly_sizeopt_t` and `poly_rootopt_t` (their scratch lives in those objects; a scan found no other shared state in stage 2 or the numerical helpers it calls). Stage 1 passes an array of per-thread callback data, and the pool's thread id selects the worker. Shared outputs — the `.ms` and `.p` files, the best-polynomial list `save_poly`, and the measurement log — are written under one mutex. The file-driven paths (`-nps` from `.m`, `-npr` from `.ms`) submit lines to a pool of the same workers.

Test	1 thread	8	16	31	Output
`-np`, one coefficient	809.5 s	—	58.8 s	35.6 s	376 survivors, same
`-nps`, 200,000 hits from a file	54.2 s	7.0 s	—	—	27 survivors, same
`-npr`, those 27	3.6 s	2.2 s	—	—	169 scored, same

4.2 A pool freed with work in it

`stage1_sieve_data_free` freed the stage 2 pool without draining it, so whatever was still queued at the end of the search was lost. It showed once all of a coefficient's survivors were delivered at once (§6): of 2064 polynomials, 1369 arrived. The pool is now drained first, unless the search was interrupted.

5 End to end: which search to keep

Whether the extra hits help is a question about the final polynomials. Three builds ran `-np` for 30 minutes each from coefficient 100000: the unmodified program, the fixed program with msieve's default search (special-q with at least d roots), and the fixed program with the full search. In a first round no polynomial reached msieve's minimum E (3.75e-10), so a second round recorded every root-optimized score (`min_evalue=1e-12`; that also lowers msieve's Bernstein pre-filter, so root optimization ran on more candidates in all three):

30 min each	unmodified	fixed, $\geq d$ roots	fixed, full
leading coefficients searched	57	3	3
hits through stage 2	1.63 M	3.50 M	4.21 M
size-optimization survivors	397	867	848
candidates root-optimized	332	627	510
best Murphy E	2.786e-10	**2.939e-10**	2.872e-10
candidates with $E \geq$ 2.5e-10 / $\geq$ 2.2e-10	7 / 49	**16 / 100**	7 / 73

The survivor rate per hit is the same in all three (about one in 4000–4700): the 999 cap did not filter bad hits, it discarded good ones with the rest. The fixed builds give two to three times as many good candidates per hour; msieve's default search was best here and stays the default. One run each: the 3–5% differences in the best E are within chance.

6 Size optimization on the GPU

With stage 2 on 32 threads, size optimization still took about 20 s per coefficient against stage 1's 7 s, and every hit crossed from the GPU to the host only to be discarded 3999 times in 4000. perf on one thread: 95% of size optimization is double-precision minimization (`poly_rotate_callback` 52%, `minimize` 18%, `ifs_radial` 16%, line searches), under 2% GMP.

6.1 Data flow

Per stage 1 batch, on the batch's stream, `so_run_batch` (`stage1_sizeopt_gpu.c`) launches `so_collide`, which rebuilds every collision of the device `found_array` in place — the hits never leave the GPU. Candidates are collected over batches; `so_flush` then runs ten rounds of `so_minimize` (one `minimize()` call per active candidate) and `so_update` (the exact integer update), `so_finish` (projective α and the bound) and `so_gather`. Only survivors are copied back, converted to GMP and queued for root optimization; the host applies the exact bound with its own `exp`. It is on by default for degrees 4 and 5 (`MSIEVE_GPU_SIZEOPT=0` keeps the CPU path); degree 6 has its own optimizer and stays on the CPU.

6.2 Integers in CGBN

The integer steps — `handle_collision`, `pol_expand`, `check_poly`, and the rotation and translation updates of `optimize_initial` — run in CGBN, two's complement, with the signed operations of the CGBN fork (`cgbn_signed_div`/`_rem` truncate like GMP's `tdiv`). The width is chosen per coefficient: 512 bits (8 threads per instance) or 1024 (16) for $\lceil \log_2 trans_N \rceil + 16$ bits. They compute GMP's integers exactly; where GMP's intermediates exceed the width the same test is done in a residue ring:

- `pol_expand` raises d^{-1} mod P to powers without reducing them, but only their residues mod P are used: the reduced powers give the same canonical residue.
- `check_poly` evaluates the homogenized polynomial, whose P^d term alone can exceed the width; it is evaluated mod N, which is the test.
- `mpz_get_d` truncates toward zero to 53 bits, and `mpz_cmpabs_d` is exact; both are reproduced (`so_get_d`, `so_cmpabs_d`).
- If P is not invertible mod $d\,a_d$, GMP leaves the destination unchanged, which holds 0 there; so does the kernel.

These kernels cost 0.25 s of GPU time for 1.78 M collisions.

6.3 Doubles, bit for bit

The minimization is msieve's own code — `minimize`, `bracket_min`, the Brent line search, `poly_rotate_callback`, the `ifs_*` norms — copied operation for operation and built with

`-fmad=false`. It then produces the same doubles as the CPU code *compiled without floating-point contraction*. That condition matters: GCC in its default GNU C mode fuses $ab + c$ into FMA, and the size optimization is sensitive to it — on the c120 coefficient the same source finds 232 survivors with contraction and 235 without, a few polynomials ending on different rotations. `-std=c11` turns contraction off, so the C11 build (§12) is the reference; its CPU path and the GPU path give identical polynomial lists.

6.4 Making the minimization fast

The first kernel (one thread per candidate, msieve's nested calls) took 48.6 s of GPU time where 32 CPU threads needed 20.5 s of wall time. Counting evaluations in an instrumented copy: 13.4 billion objective evaluations in 11.5 million `minimize()` calls, 1163 per call on average, very uneven, and a warp efficiency of 0.337. FP64 on this card runs at 332 GFLOP/s without FMA (44 G divisions/s), 1/64 of FP32. The steps, all with identical results:

Kernel version (c120 coefficient, wall time of the run)	Wall
one thread per candidate, nested calls	50.2 s
threads take candidates from a counter	58.9 s
`minimize()` as a resumable state machine	50.5 s
+ 1–2 blocks per SM instead of 8–16 (state was in local memory)	32.2 s
specialized on the degree (fixed-size arrays)	23.1 s
no local memory: direction selection by bit blends	22.3 s
batches collected before optimizing	**15.6 s**

The state machine. `so_mz_step` runs one search until it needs the objective at a point, stores the point and returns; the caller evaluates and calls again with the value. The kernel's loop evaluates for all active lanes at a single call site, so the warp computes the objective together whichever part of the search (bracketing, Brent steps, Powell steps) each lane is in. Every arithmetic operation is the original's, in the original order.

Local memory. The state (point, directions, line-search variables) was 896 bytes per thread in local memory, far more than the L1 cache once 8–16 blocks per SM were resident: fewer blocks were faster, the first sign. Making the degree a template parameter made the arrays fixed-size, but the state stayed in memory: the compiler turned the unrolled "select row `line_dir`" back into a computed index, which forces the whole structure into memory. Selecting by a blend of the bit patterns prevents that and is exact for every value, NaN and infinities included:

```
__device__ __forceinline__ static double
so_sel(double a, double b, bool c)       /* c ? b : a */
{
        long long m = -(long long)c;
        return __longlong_as_double((__double_as_longlong(a) & ~m) |
                                    (__double_as_longlong(b) & m));
}
```

The kernel then uses 202 registers and no local memory; launches are sized from the occupancy API to the resident threads (more blocks only queue).

Batching. At 140 digits a stage 1 batch has about 7000 collisions, a third of what the GPU runs at once, and each of the ten rounds waits for its slowest search. Candidates are now collected until eight times the resident count (slot = batch base + collision index), and at the end of each coefficient: the c140 coefficient went from 38.1 s to 12.1 s, c120 from 22.3 s to 15.6 s.

What did not help. Register caps with `__launch_bounds__` (168, 128, 80 registers: 31.6, 34.3, 37.8 s; spills). Reusing the last value when a search asks for a point that rounds to the same integers as the previous one — 42% of evaluations do, and the objective depends on its point only through $\lfloor x + 0.5 \rfloor$ — but the extra steps per lane diverged and duplicated the step code: 48.1 s. Evaluating the objective twice showed it is about 85% of the kernel; the kernel is FP64-bound.

6.5 Results

Input	32 CPU thr.	GPU	Survivors
c120, one coeff. (1.78 M collisions, 512 bits)	20.5 s	**15.6 s**	235, identical
c140, one coeff. (0.90 M, 1024 bits)	14.8 s	**12.1 s**	155, identical
c95, degree 4, coefficients 1000–3000	1.6 s	**1.5 s**	2064, identical

On a GPU with full-rate FP64 the minimization would be several times faster; on this one the gain is that the host's cores are free for root optimization, which stays on the CPU.

7 Root optimization on the GPU

At 155 digits root optimization dominates: one leading coefficient (`runs/ropt/c155r.ms`) gives 6,273 polynomials that pass size optimization, and msieve root-optimizes them in 195.6 s on 32 CPU threads (0.99 CPU-seconds each). For one polynomial, about 40% of the time is `optimize_basic()` (the two-variable minimization of the translated, skewed norm), 20% the line sieve, and the rest Murphy E and root properties of the best rotations.

7.1 Step 1: msieve's code, calls on the GPU

`root_sieve_gpu.c` runs `poly_rootopt_run` in coroutines (`ucontext`) on a number of threads (`MSIEVE_ROOTOPT_THREADS`), each with a number of slots (`MSIEVE_ROOTOPT_SLOTS`); inside a coroutine `optimize_basic()` queues its call and yields, and each thread answers its coroutines' calls in one batch (`so_basic_<degree>`, the state machine of §6 with two variables and the translation objective). The results were identical, but 64 polynomials took 449.6 s against 11.6 s. Each polynomial makes about 47,000 calls *one after the other*: `compute_line_size` finds a sieve line's ends by doubling and bisection, each line's search starts from the previous line's result, and the sieve loops run in order. One `minimize()` takes about 5 ms on a GPU thread (a thousand objective evaluations, each waiting for the last) against about 20 μs on a CPU core.

7.2 Throughput is not the problem

Recorded inputs (391,517 calls from eight c155 polynomials) in one launch: the GPU does 3.27 M calls/s, 32 CPU threads 2.96 M/s, with identical results. With enough independent calls the GPU is as fast as the whole CPU even on this card, whose double precision is 1/64 of single (a data-centre GPU has 1/2). What has to change is the depth: the number of rounds of dependent calls per polynomial.

7.3 The batched root sieve

`root_sieve_batch.c` keeps msieve's searches and changes where they start and when they run (`rs->batched`, per run):

- `compute_line_sizes()` runs many `compute_line_size` searches in lockstep, both ends of each at once, as state machines; each visits the original's points (checked on about 7,000 searches: no difference). Its calls go through `optimize_basic_batch()`.

- The norm bounds (about 50, from 1.01× the size-optimized norm in steps of 1.1) are searched at once, each from the first bound's starting interval instead of the previous bound's result.
- Degree 5: the x-line sizes of the y-blocks (about 900 per polynomial) are searched in three rounds — every 64th block from the original's first interval, then every 8th, then the rest, each from the sizes of its nearest searched neighbours, interpolated — instead of each from the previous block.
- Lines are kept (`rs_line_add`) and sieved 128 at a time, and the candidates of all lines (up to 25 a line) are scored 2048 at a time.

A polynomial now asks for about 300 calls per request and a few hundred requests in all. On the CPU (`MSIEVE_ROOTOPT_BATCHED=1`) it runs at about the original's speed; its purpose is the GPU, and checking the GPU against it.

7.4 The line sieve on the GPU

A line is scored at positions $x = 0, \ldots, 8192k - 1$: the score is the sum over the 25 primes below 100 of $T_p[x \bmod |T_p|]$, where each prime's table (at most 1000 entries) holds the contributions of its powers' roots for that line. msieve fills the tables on the CPU per line; shipping them would move 22 MB per polynomial. `ls_sieve` (in `sizeopt_core.cu`) takes one line per block of 512 threads and builds the tables in shared memory from the polynomial's roots (about 6,400) and the line's coordinates (five 64-bit numbers; `line_sieve_core.h` holds the residue arithmetic, inline code shared with the CPU path). Thread t scores $t, t + 512, \ldots$ (about 230,000 positions a line) with 25 running table indices, and keeps a position when it beats a threshold, the 25th best key kept so far; when the kept list fills it is cut back to the 25 best by rank. Keys order by score and then by lower position, so the result is the same whatever order the threads find positions in; msieve's heap keeps an arbitrary one of equal scores, and the CPU path of the batched root sieve uses the new rule too. The kernel uses 128 registers and no local memory. Reading the line's x base with `mpz_get_si` exposed an msieve bug: `sieve_x_t.x_base` was never initialized (GMP grows a zeroed `mpz_t` on its first write, so msieve's own code never noticed); it is now.

7.5 The engine

Each thread's slots are in two groups with a stream each: the thread collects one group's batch, runs that group's coroutines until each asks for calls or a line sieve, and launches the group's next batch, then does the same for the other group, so the CPU work of one overlaps the GPU work of the other. Three things mattered in getting there:

- Growing the pinned batch buffers step by step meant 1,982 reallocations; `cuMemFreeHost` waits for the GPU and holds the driver lock the other threads' copies need (500 s summed over threads). The buffers are now sized once.
- Threads still setting up found the queue empty: one thread took 39 of 155 polynomials. Slots are set up on first use, outside the queue lock, and a thread takes at most its fair share of the jobs, queued and running.
- On this card the GPU becomes the limit, and while stage 1 runs it is busy anyway. A thread does a share of its batch's calls itself while the GPU runs the rest; the share rises when the thread had to wait for the GPU and falls when it did not (`MSIEVE_ROOTOPT_CPU_SHARE`), so it tends to zero on a fast GPU. The results do not depend on it.

7.6 Results

c155, one coefficient (6,273 polynomials)	Time	Saved	Best E
msieve, 32 CPU threads	195.6 s	49	2.510e-12
GPU, `optimize_basic` only	169.6 s	50	2.510e-12
+ line sieve, buffers, fair shares, CPU share	107.0 s	50	2.510e-12
+ root properties on the GPU (CGBN)	**102.6 s**	50	2.510e-12
(+ Murphy E on the GPU, this card)	170–183 s	50	2.510e-12

The 50 polynomials saved are the 49 msieve saves and one more. With every score recorded (`min_evalue=1e-12`) the GPU and the CPU batched root sieve agree bit for bit at 95 (degree 4), 120, 140 and 155 digits; against msieve the best E is the same at each size, and 97–100% of the scored polynomials are identical (the others differ because a line's end moved within the search tolerance or a tie went the other way). With few polynomials in flight (235 for a 120-digit coefficient, 155 at 140 digits) the time is set by one polynomial's few hundred rounds, and GPU and CPU take about the same time. Degree 6 batches the norm bounds, lines and candidates but not its y- and z-line searches. Tested on 16 degree-6 polynomials of RSA-1024 (309 digits): GPU and batched CPU identical, but 1061 s on the GPU against 466 s on the CPU, the y- and z-line searches sending about one `optimize_basic()` call per round trip (8.0 million batches for 9.4 million calls).

7.7 The last step: root properties and Murphy E

For each of the up to 200 rotations in a polynomial's root heap, `optimize_final` computes the root properties α (rotated coefficients reduced modulo the 303 primes below 2000, roots counted mod p, special roots lifted mod p^k), a Bernstein score as a filter, and, for the few that pass (about 9 per polynomial at 155 digits), maximizes Murphy E over skewness and translation: `minimize()` of an adaptive double-exponential integral of Dickman ρ terms, about 220 integrals per candidate. With the rest on the GPU these were the CPU's main work: Murphy E about 45%, α 14.5% (of which GMP, the `mpz_fdiv_ui` reductions, 1.4%). `optimize_final_batch` now does all of a heap's α at once, the filter on the CPU, then all of its Murphy E optimizations at once, each step through a hook.

Root properties. `alpha_512`/`alpha_1024`: one block per rotation. CGBN forms $b = a + (x + yX + zX^2)(PX - d)$ exactly and reduces each coefficient with `cgbn_rem_ui32` (floor remainders, as `mpz_fdiv_ui`); one thread per prime then runs ports of `poly_get_zeros`, `get_zeros_rec`, `poly_get_zeros_and_mult` and `get_root_freq`, and thread 0 adds the terms in the CPU's order with the host's $\log p$. The order of a polynomial's roots decides the order of the additions, so the gcd and `mp_modsqrt_1` are ported as they are ($x^n \bmod f$, being unique, is written plainly). The results are the CPU's bit for bit at 95, 120, 140 and 155 digits. Two faults, both reported as illegal addresses: CGBN loads need 16-byte aligned limbs, and the recursion needs a 6 KB device stack. This is integer work, which this card has to spare: 107.8 s → 102.6 s.

The root sieve's setup. `init_sieve` reduced the coefficients, P and d with GMP modulo each of the 52 prime powers below 1000 and listed the roots of each power, about 6,400; the line sieve then had them sent with every batch of lines. `rs_setup_<bits>` does it in one block per polynomial: CGBN residues, then the threads test the residue classes of each power and compact the roots in the CPU's order. The roots stay in a per-slot device area that `ls_sieve` reads; the host gets one copy for its lattice choice, and the number of classes adding to the sieve bias, which it adds up in the original order: roots and bias are the CPU's bit for bit. Two engine lessons came with it: the setup's pinned buffers, grown at startup, cost 10% (batches are now capped), and every thread buffer is now allocated before any thread starts. A c155 coefficient, back to back: 105–106 s against 110–113 s with the setup on the CPU.

The lattice CRT. The degree 4 and 5 x sieves formed each line with GMP — the lattice CRT $r = (r_1 c_0 + r_x c_1) \bmod L$, $x_0 = \lfloor \ell_{\min}/L \rceil L$ (truncated), $y = y_0 + r_y + L_y k$ — and after the GPU line sieve the host formed each candidate ($x = x_0 + r + L\,pos$ and its rotated polynomial) and sent it back for `optimize_basic`. Now a line reaches the GPU as its parts and the run's constants (the CRT constants in 64- and 128-bit integers instead of `mpz_invert`); `ls_sieve` forms the numbers in CGBN, takes its residues from them and makes the candidates, `mpz_get_d`'s truncation included: identical results. Where double precision is fast their `optimize_basic` follows on the GPU; on this card that cost 8–12% (those calls cannot be shared with the CPU), so the candidates' polynomials come back to the shared batches. One lesson: the sieve loop keeps about 75 values per thread, and the larger line structure and inlined CGBN code pushed it into spills, 50% slower per position. The tables' layout, the same for every polynomial, now sits in constant memory: 64 registers instead of 128, and the same speed as before.

The lattice choice. By default (`GPU_LATTICE=1`, `RS_GPU_LATTICE`; `GPU_LATTICE=0` keeps the CPU code), the lattice choice of degrees 4 and 5 runs on the GPU too, on the roots the setup left there: `lt_xy` (one thread per prime of the xy lattice, its 2-D sieve and hit list) and `lt_x` (the x-line choice: threads take the (lattice, y block) items, each block's offsets computed directly instead of one after the other, the combinations in `compute_lattices`' order, and thread 0 fills the 300-entry heap as `save_xline` does). The roots then never go to the host; a CPU path that needs them fetches them. The results are identical, but this work is 0.1% of the CPU time, and on the RTX PRO 4500 the extra round trip per choice made a c155 coefficient 4% slower (110 s against 105–107 s); it is the default anyway, for the target hardware (an RTX PRO 6000 Blackwell or larger).

The kept polynomials. The polynomials that pass are rotated and translated exactly on the GPU (`rf_final`, CGBN at 1024 bits, one 32-thread group each: the rotation, then the translation by Horner's rule with binomial weights), and with Murphy E on the CPU their inputs come from `bs_size_<deg>` rather than being rebuilt from `mpz`. Rotations travel through the heap as 64-bit integers (`mpz` only for one that does not fit), so no GMP runs between the GPU steps. Results identical; same speed (111.2 against 110.4 s over five alternating runs), 0.5 GB less memory.

The per-run lattice numbers. For degrees 4 and 5 the numbers of a run of lines are machine words too: y_{base} as a 64-bit integer (as `mpz_set_d` and `mpz_mul` make it; GMP only if it does not fit), the two lattice factors as 64-bit words, the lattice size as a double truncated as `mpz_get_d` does, and the divisibility tests of the x lattice choice by `%`. `ls_sieve` forms the 128-bit lattice size from its factors in CGBN, and `lt_x` takes y_{base}'s residues from the word. The `mpz` forms are made only when CPU code needs them (`rs_xy_mpz`, `rs_x_mpz`). Host GMP is now left for the CPU fallbacks and the callback. Same speed (168.2 s for a c155 coefficient with the defaults, 110.3 s with both FP64 paths off).

Degree 6. Its plane constants (the xy lattice size $L_{xy}L_{xyz}$, the CRT constants, y_{base} and the plane's residue classes) and its x lattice's (the size, CRT constants from `rs_lattice_crt`) are 64- and 128-bit words when $L_{xy}L_{xyz} < 2^{63}$; GMP otherwise, as before. Its x-lines then go to the GPU as parts ($y = y_{\text{base}} + r_y + L_{xy}L_{xyz}k$, the lattice CRT and x_{base} formed in CGBN by `ls_sieve`): before, most of them did not fit 64-bit coordinates and were sieved on the CPU (55,802 lines on the GPU now, 1,435 before). So that the batched CPU sieve stays the reference, it takes parts lines too, forming the same numbers with GMP; it used to sieve the big lines with msieve's own line heap, which breaks ties differently and gave 6 different polynomials in 3,200. With that, GPU and batched CPU are identical on 16 degree-6 polynomials of RSA-1024, and degrees 4 and 5 are unchanged.

Degree 6 batched. Its line searches were sequential: per polynomial 3,700 y-line searches of z-blocks (148,000 `optimize_basic()` calls), each starting from the previous block's result, 50 first x-line searches of planes and 1,806 x-line searches (135,000 calls together), so each call was about one GPU round trip. The z-blocks now go in strided rounds with interpolated starts, the same code as degree 5's x-lines (`rs_lines_strided`), and the x-line searches of all planes of a z-lattice run together (`sieve_x_run_deg6_batch`: first searches, then each plane's x lattice sieve, then all x-lines, then the lines in the original order; same starts, same results). 16 RSA-1024 polynomials: round trips 8.0 million → 8,346, 1061 s → 417 s (CPU 466 s); GPU and batched CPU identical; against the unbatched searches 3,183 of 3,200 polynomials the same and the top 200 E values equal.

The Bernstein filter. `alpha_<bits>` also writes each rotated polynomial as double-double, converted from CGBN's limbs as `dd_gmp2dd` does (top word first, $d \leftarrow d \cdot 2^{32} + w$: the same bits). `bs_size_<degree>` then takes the rotations whose α passes, one warp each: lane 0 finds the roots with a port of `find_poly_roots` (Jenkins–Traub in double precision, Newton polishing in double-double complex), and the lanes share the integral of $|R(x)A(x)|^{-2/(d+1)}$ with the same integrator as Murphy E, now generic over the integrand. On 19,029 rotations the scores were within 1 ulp of the CPU's (48% identical) and the filter decided the same every time. The host now does big-integer work only for the rotations that pass (about 9 of 200). With Murphy E on the GPU, not even that: the kernel applies the bound itself and writes each survivor's Murphy E input (coefficients, root scores, roots) into a device area that `mf_opt` reads through an index list, so the heap goes from rotations to Murphy E without its numbers crossing to the host; GMP is left for the polynomials that are saved. The results equal those of host-built inputs exactly.

Murphy E. `mf_opt_<degree>`: one warp per candidate. All lanes step the same `minimize()` state machine; each integral's subintervals are refined as in `de_run`, and the points of each trapezoid level are spread over the lanes. The polynomial's roots, which split the integral, are found once on the host; a translation by t moves them by t. The GPU's `sin`, `cos` and `log` differ from the C library's in the last bit, so E agrees to a median relative $2 \cdot 10^{-13}$ (at most $6 \cdot 10^{-9}$); where E is flat in the translation, the two stop at different integers. Alone, the kernel does 1,512 candidates/s against 1,263 for 32 CPU threads. In the pipeline, on this card, it made the run slower (170–183 s, also with the batches on their own low-priority streams): the GPU's double precision was already the limit, and a Murphy block occupies a multiprocessor for a quarter of a second. It is on by default anyway, with the line candidates' `optimize_basic()` on the GPU, because the defaults are for the target hardware (an RTX PRO 6000 Blackwell or larger), not this card; on a small card `MSIEVE_GPU_MURPHY=0` and `MSIEVE_GPU_LINECAND=0` are faster. Both used to follow the FP64 ratio (on only at $\geq 1/4$ of single precision), which the RTX PRO 6000 Blackwell, at 1/64, fails too. Here a c155 coefficient takes 168 s with the defaults (5.6 GB) against about 111 s with both off (8.3 GB); the results are the same.

More than one GPU. Polynomial selection uses every GPU unless `-g` names one (`MSIEVE_ALL_GPUS`). Stage 1 runs thread t on GPU t mod n, one to four threads per GPU, each GPU's memory budget shared by its threads; the size optimization follows on the same contexts. Root optimization has one engine per GPU (context, kernels, tables, threads) behind one job queue; a polynomial stays on the GPU whose thread took it, and jobs are shared out per GPU first, then per thread, so that a few polynomials still spread over the cards. No data moves between GPUs: the work is independent per leading coefficient and per polynomial, so NVLink and SLI play no part. `gpu_init` recorded at most four devices and wrote past its table on larger machines; it now records up to 64. `MSIEVE_SIMULATE_GPUS=`n runs n logical GPUs on the card(s); results are identical with 1–4 of them.

Several nodes. Built with `MPI=1`, polynomial selection runs on every MPI process: process r of R searches the k-th leading coefficient the a_d sieve accepts when $k \bmod R = r$ (so all of them search the same low region, and a deadline cuts them alike), and `-nps`/`-npr` split their input by line. A node's processes divide its GPUs and CPUs; processes sharing a card divide its stage 1 memory budget (three on one card ran out of memory before). Rank $r > 0$ writes its own `.mpiNN` files; at the end rank 0 collects them and every process's best polynomial over MPI, so no shared file system is needed, and writes the `.fb`. Tested with 1–4 processes on one machine: `-np`, `-np1`, `-nps` and `-npr` give the same files and the same best polynomial as without MPI.

8 Linear algebra

GPU builds run the block Lanczos iteration on the GPU. The matrix and all Lanczos vectors stay in device memory; vector handles are device addresses. The matrix is stored twice, by rows for Ax and by columns for $A^T x$, a warp per row or column XORing the gathered 64-bit block entries, and its dense rows are bitmaps (a warp takes 32 columns at a time, each lane one dense row). The 64×64 products $x^T y$ and $y \oplus xX$ use registers and nibble tables in shared memory. msieve's CPU backend stays and `lanczos_dispatch.c` chooses per matrix (`MSIEVE_GPU_LA`, `MSIEVE_GPU_LA_MIN`, GPU memory).

cuSPARSE was measured first. It has no GF(2) arithmetic; int8 SpMM with parity and the deprecated custom-operator `cusparseSpMMOp` were 8–12× slower than the custom kernel on a 4M × 4M matrix with 240M nonzeros.

Several GPUs use one MPI process each on msieve's existing MPI grid. With a CUDA-aware MPI (HPC-X here) the per-iteration grid operations hand the device vectors to MPI, their local XORs run on the GPU, and the GPU is synchronized before each MPI call; otherwise they stage through pinned host buffers. The rare result gathers always stage (HPC-X gave wrong data for a gather mixing device and host buffers). On one shared card, 600k on a 1×2 grid: 36.4 s CUDA-aware, 40.6 s staged. An NCCL transport (build option, off by default) runs the same grid operations on the device vectors, stream-ordered: `ncclAllGather` for the gather, and rings of `ncclSend`/`ncclRecv` with the GPU XOR kernel for the XOR reductions (NCCL has no XOR). NCCL needs one GPU per process, so it could not run here; the ring logic was checked through an MPI transport on host buffers on six grids, and the fallback (the processes compare GPU UUIDs first) on two. On synthetic NFS-like matrices (every dependency checked): 200k in 3.6 s (CPU 56.4 s, one thread), 600k in 15.8 s (CPU, 16 threads, 116.0 s), 2M in 157.5 s, 5.0 ms per iteration; each product at about 760 GB/s of the card's 896 GB/s. MPI grids 1×2, 2×1, 2×2 and 1×3, checkpoint and resume, and QS are correct. Fixed on the way: the checkpoint interval, calibrated with whole seconds, collapsed to its minimum on the GPU (now at least 5 s of timing), and `make clean` left GPU-only objects behind.

Synthetic NFS-like matrix (60 nonzeros per column)	GPU	CPU, 16 threads
200k × 200k, 12M nonzeros	3.6 s	56.4 s (1 thread)
600k × 600k, 36M nonzeros	15.8 s	116.0 s
2M × 2M, 120M nonzeros	157.5 s (5.0 ms/iteration)	—

Grid operations, all processes on one card	CUDA-aware MPI	staged
600k matrix, 1×2 grid	36.4 s	40.6 s
200k matrix, 1×2 grid	11.2 s	8.4 s

Sparse product, 4M × 4M, 240M nonzeros	Time per product
custom XOR kernel (warp per row)	3.4–3.8 ms
cuSPARSE int8 SpMM with parity, `cusparseSpMMOp`	29.5–39.6 ms

VBITS 128 and 256 are correct but not faster (16.3 and 17.1 s at 600k).

9 Line sieving on the GPU

GPU builds run msieve's line siever (`-ns`) on the GPU, and find exactly the candidates and relations of the CPU siever (`gnfs/sieve/sieve_line.c`). The GPU (`gnfs/sieve/sieve_core.cu`) takes batches of up to 16 lines (values of b) with one logarithm base, recomputed every 100 lines as the CPU does:

- `ls_starts`: each factor-base entry's first position on each line, with the CPU's skip rules and its compression of even lines;
- `ls_bucket`: per 2^{20}-position chunk, the hits of the primes above 16,384 sorted into buckets per segment;
- `ls_segment`: a block per 16,384-position segment, sieving in shared memory (both sides in one 32-bit word; primes below 256 and the prime powers by position, the others by entry with atomics, then the buckets). It computes the cutoffs as `sieve_line.c` does, from the exact bit sizes of the norms (a 384-bit evaluation at each window's ends, the same 2/3-large-prime choice). It then applies the gcd test and resieves the primes ≥ 1000 for each candidate.

Host threads (`sieve_line_gpu.c`) trial-divide the small primes, divide out the resieved ones and check the cofactors as `do_one_factoring()` does; the cofactors go to msieve's batch factoring. While the host factors one chunk, the GPU sieves the next. The batch factoring's remainder tree (`common/batch_factor.c`) now runs on several threads, for the CPU siever too. The GPU siever falls back to the CPU for degree above 8, coefficients of 255 bits or more, norms that could exceed 382 bits, or less than 74 KB of shared memory per block. Settings: `MSIEVE_GPU_SIEVE=0` (CPU siever), `MSIEVE_SIEVE_THREADS`, `MSIEVE_BATCH_THREADS`, `MSIEVE_SIEVE_DUMP` (both sievers write every candidate, for comparison).

Three bugs were found on the way. The host threads called `eval_poly`, which keeps its temporaries in the shared polynomial (a `realloc` crash, and one candidate lost); they now have their own. At $b = 1$ a single line had about a million candidates (80,000 in one chunk), so the candidates go to the host after every chunk, and a chunk that still overflows is redone line by line. Nothing stopped a large polynomial from overflowing the 384-bit norm evaluation; a host bound on the norms now sends it to the CPU.

9.1 Results

c95 (degree 4, polynomial from `-np "min_coeff=1000 max_coeff=3000"`), RTX PRO 4500 Blackwell, 32 CPU threads. Relations are compared with each side's primes sorted.

b range	Candidates (GPU vs CPU)	Relations (GPU vs CPU)
1000–1009	295,216, identical	1,788, identical
1–300	40,464,202, identical	345,088, identical
1000–2000	—	133,558, identical

b range	CPU siever	CPU, threaded batch	GPU siever
1000–1009	—	—	0.4 s
1–300	107.0 s	97.8 s	**5.8 s** (18×)
1000–2000	1000–1921 in 120 s (stopped)	—	**4.8 s**

GPU siever version (wall time)	b 1–300	b 1000–2000
first: factored per batch, serial batch factoring	12.3 s	10.2 s
threaded batch factoring	7.3 s	7.8 s
GPU sieves the next chunk while the host factors	**5.8 s**	**4.8 s**

Version	b range	Total	Waiting for GPU	Host factoring	Batch factoring
before threaded batch	1–300	11.8 s	0.8 s	4.0 s	6.7 s
before threaded batch	1000–2000	9.7 s	2.5 s	3.8 s	3.1 s
final	1–300	5.2 s	0.2 s	3.2 s	1.4 s
final	1000–2000	4.2 s	0.4 s	2.6 s	0.8 s

The breakdown is the siever's own log line, without startup. The CPU siever's profile before the port: sieving (`fill_one_block`) 42 %, resieving 21 %, trial division and GMP about 30 %. On the GPU the host is the limit: host factoring of 40 million candidates (b 1–300) or 22 million (b 1000–2000) is about 60 % of the time and the GPU is mostly idle on these short ranges; over a whole factorization it is the other way round.

The whole c95. `-ns "1,100000000"` (the siever stops when it has the relations filtering wants), then `-nc` (filtering, the linear algebra on the GPU, the square root):

Step	Result
Sieving	b 1 to 15,005,348, **6,308,499 relations in 33,210 s** (9 h 14 min)
Siever's log line	waiting for the GPU 21,951 s, host factoring of 14.18 billion candidates 10,737 s, batch factoring 717 s
Relations checked	6,308,499 valid, 0 invalid (each listed prime divides its norm, the rest has only primes below 256)
Filtering	0 duplicates, 6,472,801 unique relations (with the free relations), matrix 428,944 × 429,170
Linear algebra (GPU)	35.4 M nonzeros, 6,784 Lanczos iterations, 33 dependencies
Square root	p47 × p48 (§10)
Postprocessing	120 s

Over the whole run the GPU is the limit, not the host: the candidates per line fall as b grows while each line costs the GPU about the same, so two thirds of the time is waiting for the GPU (2.2 ms per line on average). The lattice siever (§10) gets the relations for this number in 43.7 s. This run predates the fix to msieve's batch factoring (§10), which could drop a few relations per run.

Several GPUs or nodes: one GPU per process; as with the CPU line siever, processes get separate `-ns` ranges.

10 Lattice sieving on the GPU

msieve has no lattice siever of its own. GPU builds add one, and `-ns` runs it there: `-ns "q0,q1"` sieves special-q lattices on the algebraic side for q from q_0 to q_1 (above the factor base; without a range from the factor base's end, or from where the last run stopped, kept in `<savefile>.lattice`). `-ns "line ..."` runs the GPU line siever (the range is then one of b), `MSIEVE_GPU_SIEVE=0` msieve's CPU line siever; if the lattice siever cannot run (no usable GPU), the line siever runs. Like the line siever it stops when it has the relations filtering wants, and it writes them to the savefile in msieve's format, so `-nc` runs on them unchanged. Each root r of $f \bmod q$ is one lattice, Gauss-reduced with the polynomial's skew: $a = a_0 i + a_1 j$, $b = b_0 i + b_1 j$, $-I/2 \le i < I/2$, $0 \le j < J$, with $I = 4096$ and $J = 2048$ by default. The sieve value is one 32-bit word per position, the rational log sum in the low 16 bits and the algebraic one in the high 16 (16 log units per bit, so a sum cannot reach 2^{16}).

The GPU (`gnfs/sieve/lasieve_core.cu`) sieves a batch of lattices (64 here, sized to free memory):

- `la_roots`: each factor-base entry's root in each lattice, $\rho = -v/u \bmod p$ with $u = a_0 - Rb_0$, $v = a_1 - Rb_1$ (projective roots: $u = b_0$, $v = b_1$). An entry with $u \equiv 0$ divides the norm at every position of the rows $j \equiv 0 \pmod p$; it is marked and handled per row;

- `la_rowlogs`: for every row of every lattice, the logs of those $u \equiv 0$ entries whose p divides j;
- `la_large`: the primes $p \geq I$ with the Franke–Kleinjung walk (a reduced basis per prime, one step per hit), adding into the lattice's sieve in global memory, one launch per lattice so that the atomic adds stay in L2;
- `la_tile`: a block per 4 rows in shared memory. It adds the primes $32 \leq p < I$, sets each run of 32 positions' thresholds from the norms there (msieve's 2/3-large-prime choice), lowered by the row's $u \equiv 0$ logs and by the expected contribution $\sum \log p \cdot p/(p^2 - 1)$ of the primes below 32, which are not sieved. It applies the gcd test, stores each candidate in a list and a GPU hash table, and resieves the small primes ≥ 256;
- `la_large` again, resieving: a hit on a candidate (a bitmap check, then the hash) adds the prime to that candidate's own list;
- `la_filter`: the rational side's cofactor test on the exact 128-bit norm, which removes two thirds of the candidates;
- `la_cofactor` (CGBN, 256 bits, 4 threads per candidate): the rest of the host's tests. The algebraic norm by Horner's rule (two's complement modulo 2^{256}, exact as the host bounds each batch's norms below 2^{253}), divided by q, the listed primes, the row's prime and the primes below 256 (12 groups whose products fit 32 bits); msieve's algebraic large-prime bounds; and the base-2 Fermat test on both cofactors (at 128 bits). Only 2.8 % of the candidates past `la_filter` are copied to the host.

Host threads (`gnfs/sieve/sieve_lattice_gpu.c`) divide out q, the listed primes, the $u \equiv 0$ prime of the candidate's row ($j < 2^{13}$ has at most one prime factor ≥ 256, `la_jprime`) and the primes below 256, check the cofactors as the line siever does (large-prime bounds, a Fermat test) and send the rest to msieve's batch factoring. They factor batch k while the GPU sieves batch $k + 1$ (two pinned buffers, one host job at a time); batch factoring runs in a background thread on a second relation batch (its relations and the host threads' reach the savefile under one lock). The stop test adds the relations the batched cofactors not run yet should give, at the rate seen so far, then drains everything and checks. msieve's filtering removes the relations two special-q both find. Settings: `MSIEVE_LATTICE_LOGI` ($\log_2 I$, 12), `MSIEVE_LATTICE_TINY` (32), `MSIEVE_LATTICE_FILTER=0` (no rational test on the GPU), `MSIEVE_LATTICE_COFACTOR=0` (no `la_cofactor`), `MSIEVE_LATTICE_BATCH`, `MSIEVE_LATTICE_CHECK=1` (the first lattice's sieve sums against a direct computation on the host), `MSIEVE_LATTICE_VERIFY=1` (every cofactor checked for a factor-base prime ≥ 256 the GPU did not report), `MSIEVE_BATCH_DUMP` (the cofactors sent to batch factoring, for `test/sieve/batchreplay.c`), `MSIEVE_SIEVE_THREADS`.

Bugs found while building it: `init_one_fb` takes over the factor base's entries (the siever read freed memory; it now uses `sieve_t`'s copy); the last batch was dropped when q_1 was not prime; the rational polynomial's constant has 76 bits on the c95, so the filter's norm is 128-bit; the first relation check rejected 6 relations with a squared large prime, which msieve's batch factoring lists twice (the checker was wrong, not the relations).

10.1 Three bugs that lost relations

The first version of this section reported 25,468 relations on the test range and 6,374,921 for the c95; once in about 20 runs of the test range, 9 relations were missing. The investigation found three bugs:

Bug	Effect	Fix
Entries with $u \equiv 0$ neither sieved nor resieved	their prime stayed in the cofactor, which batch factoring (primes above the factor base) cannot split: 105 such cofactors on the test range (110 relations more after the fix), 30,127 relations on the c95	`la_rowlogs` adds their logs per row; `la_filter` and the host divide out j's prime
At $i = 0$ the large primes with $\rho = 0$ hit the whole column; the FK reduction cannot walk them	the candidate at $(j = 1, i = 0)$ kept them: 6 on the c95	that position goes to the host untested, which scans the factor base
msieve's batch factoring (`common/batch_factor.c`, upstream) skipped the relations of a remainder-tree leaf whose remainder was zero	a zero remainder means the leaf's cofactors divide the prime product, i.e. all are smooth: exactly the relations wanted. It depends on which relations a batch puts together, in the sievers on thread timing: 7–9 relations in about one run in six	the leaf is checked anyway ($\gcd(0, n) = n$)

The third bug affects every msieve siever that batches, the line sievers included. How it was found: in a run that lost relations every count matched a normal run, the lost relations all came from batch factoring and clustered in neighbouring lattices, and `MSIEVE_LATTICE_VERIFY` reported no missed prime. `MSIEVE_BATCH_DUMP` writes the cofactors a run sends to batch factoring; `test/sieve/batchreplay.c` feeds them through msieve's batch factoring in shuffled orders. 2 of 20 orders found 3,624,662 relations instead of 3,624,670, and an instrumented remainder tree showed 2 zero-remainder leaves holding exactly those 8. With the fix all 20 orders give the same 3,624,670 relations. `MSIEVE_LATTICE_VERIFY` found the first two bugs directly; `MSIEVE_LATTICE_CHECK` had passed because it skipped the same $u \equiv 0$ entries as the GPU.

One hardening on the way, not the fix of any observed loss: `la_tile` looks hash keys up while other blocks still insert theirs, so those lookups read from L2 (`__ldcg`); a plain load may return a line the SM cached before another SM filled a slot. `la_large`'s lookups, in a later launch, keep plain loads (`__ldcg` everywhere cost 12.6 s of GPU time on the c95).

10.2 Results

c95 as in §9, RTX PRO 4500 Blackwell, 32 CPU threads.

Correctness. `MSIEVE_LATTICE_CHECK`: 0 of 7,096 sampled sieve sums differ from the host's direct sums (q 2,200,031), the $u \equiv 0$ rows included. `MSIEVE_LATTICE_VERIFY`: no cofactor with an unreported factor-base prime, on the test range and on the c95. Every relation was checked independently (`test/sieve/relcheck.py`: each listed prime divides its side's norm, the rest has only primes below 256): 25,768 of 25,768 on q 2,200,000–2,201,000 and 6,405,054 of 6,405,054 for the whole c95. 40 runs of the test range (20 plain, 10 under CPU load, 10 under nsys) all gave 25,768 relations, and 8 runs of the whole c95 all gave 6,405,054.

The whole c95, `./msieve -n` with the c95's factor base (every stage at its default), on an otherwise idle machine:

Step	Result
Sieving	special-q 2,120,430–2,384,071, 18,038 lattices, **6,344,609 relations in 42.6 s** (other runs 43.0 s)
Siever's log line	GPU 38.6 s; 7,217,394 candidates factored on the host alongside the GPU, batch factoring 28.4 s; waiting for the host 0.1 s
Candidates	776,648,279 found, 256,003,536 past `la_filter`, 7,217,394 past `la_cofactor` (170.8 million failed its algebraic bounds, 78.0 million its Fermat test); 2,754,137 relations direct, 4,463,189 cofactors to batch factoring
Filtering	104,888 duplicates, 6,398,252 unique relations, matrix 126,032 × 126,259
Linear algebra (GPU)	10.3 M nonzeros, 1,994 Lanczos iterations, 33 dependencies
Square root	p47 889506722080817233974832947931318686826271154 13 × p48 303880173004512093573696089949868722431828017591
Total	**111.4 s** (sieving 42.6 s, postprocessing about 68 s); **67.8 s** with the postprocessing below (sieving 43.1 s, postprocessing about 24 s)

All 6,344,609 relations of the same sieving run with `-ns` are valid. Without `-n`, msieve factors a c95 with its quadratic sieve (MPQS, CPU): 46 min 40 s.

Each step's gain on the whole c95 (sieving, the siever's log line):

Version	Sieving	GPU	Host	Waiting	To the host
the three fixes (8 runs)	105.0 s	34.7 s	68.4 s	—	258,450,297
+ `la_cofactor`, 8 threads per candidate	93.0–94.2 s	47.1 s	45 s	—	7,286,121
+ 4 threads, Fermat at 128 bits, unrolled trial division, host overlapped	71.8–72.3 s	39.6 s	—	30.7 s	7,309,655
+ batch factoring in the background, stop estimate	**43.7 s**	38.9 s	—	0.1 s	7,217,394

c95	Line siever (GPU)	Lattice siever (GPU)
Sieving	33,210 s	**43.7 s** (760×)
Relations	6,308,499	6,344,609
Duplicates	0	104,888
Matrix	428,944 × 429,170	126,032 × 126,259
Lanczos iterations	6,784	1,994
Postprocessing	120 s	about 24 s
Total	9 h 16 min	**1 min 8 s** (`./msieve -n`)

"Host" is the host's factoring time where it ran after the GPU, "waiting" the main thread's wait for the host once they overlap (batch factoring 31.5, 32, 32 and 28.4 s in the four rows). The overlapped versions sieve 18,038–18,272 lattices instead of 18,212. The third row did not depend on the thread count (16, 24, 31 threads: 71.1–71.5 s): each batch run held the host job up, so the GPU waited for it. `la_cofactor` per 72 lattices (nsys): 43.7 ms with 8 threads per candidate and the Fermat test at 256 bits (19 ms of it), 36.1–37.1 ms with 4 and 128 bits; `la_filter` went from 1.8 to 4.0 ms when its primes came from a constant-memory table and back below 1.7 ms written out as constants. The lattice relations give a matrix 3.4 times smaller than the line siever's, from about as many relations.

10.3 Postprocessing on all CPUs and in memory

After the sieving work above, postprocessing was the larger part (about 68 s). Reading the gzipped savefile is not what costs: on the same relations 62.6 s from the gzipped file, 59.5 s from uncompressed text. msieve's filtering reads and parses every relation in several passes, on one

thread: duplicate removal's first pass checks each relation against its norms, singleton removal's first pass computes each large ideal's root (`find_large_ideals`, 38 % of a profile), and the matrix and every square-root dependency read their relations again. Two changes:

- **All CPUs** (`gnfs/filter/relpar.{c,h}`): a pass adds a block of 16,384 lines, which are parsed (and their ideals found) on all threads, each with its own copy of the polynomials (`nfs_read_relation` keeps scratch values in them); the pass then takes the results in order, so its output is unchanged. Used by duplicate pass 1, singleton removal's initial pass and the cycle reader (matrix, square root); `MSIEVE_FILTER_THREADS`.
- **In memory** (`gnfs/relstore.{c,h}`): when `factor_gnfs` starts a new savefile, every relation written to it (all sievers, and the free relations filtering adds) is also copied into memory as text, under the lock that orders the savefile, so the sievers do no more work. The first time filtering asks, they are parsed once on all CPUs (primality tested, as duplicate pass 1 does), and every later pass reads the parsed relations instead of the file. The savefile is written as before; another process (`-nc`), a restarted savefile, `MSIEVE_RELSTORE=0` or a store over `MSIEVE_RELSTORE_MB` (default: a quarter of physical memory) read the file as before.

c95 (log timestamps)	Dup. 1	Dup. 2	Singl. 1	In-mem.	Matrix	Sqrt/dep.	To sqrt
msieve's, one thread, gzipped file	22 s	2 s	18 s	6 s	8 s	6 s	56 s
parsed on all CPUs	8 s	2 s	8 s	5 s	8 s	5.5 s	31 s
in memory	**4 s**	<1 s	**5 s**	5 s	6 s	4 s	**20 s**

`./msieve -n` on the c95: 111.4 s before, **67.8 s** after (sieving 43.1 s; the square root needed one dependency; each has about an even chance, so it took 1–5 in these runs). The duplicates, ideals, every matrix size and the cycle weights are the same with the store as with a file-path `-nc` on the same savefile, and the file path gives the same results as before. Two ways of filling the store were measured and dropped: parsing each relation in the siever (inside batch factoring's output lock it serialized the batch threads; outside it, contention for the savefile lock cost 8 s of sieving), and listing the primes below 256 in the relations so that the parse needs no trial division (8 s more sieving).

In-memory singleton passes on all CPUs. msieve repeats them after each clique removal. Each pass now marks, on all threads, the relations with an ideal of count 1 and decrements their ideals' counts atomically; the survivors are compacted in order at the end. The relations left (every ideal in at least two) do not depend on the order of removal, and a stale count only puts a removal off to the next pass, so the result is the same (the filtering log is identical apart from the pass counts). The six rounds on the c95: 1.15–1.22 s → 0.39 s.

The NFS by default (GPU builds). msieve used its quadratic sieve (MPQS) unless given an NFS option. GPU builds now run the whole NFS without one for inputs of 80 digits or more (85 at first) when a CUDA device is there (`MSIEVE_NFS_MIN_DIGITS`; `MSIEVE_DEFAULT_NFS=0`: MPQS). From scratch, balanced semiprimes:

Digits	MPQS	NFS	Polynomial selection
80	132 s	—	(no NFS parameters below 80 digits)
85	350 s	**227 s**	159 s, one retry
90	981 s	**395 s**	339 s, one retry
95	2,800 s	**413 s**	345 s

Polynomial selection had to change first (the first runs: no polynomial at 85 digits, 1,324 s at 90, 1,130 s at 95). Its time limit counted only stage 1's own CPU and GPU time, which barely grows

while stage 1 waits for root optimization; at these sizes every size-optimized polynomial passed the stage 2 bound (1.2 million at 90 digits) and root optimization took them all (166 s ran 20 minutes). Stage 1 now also stops at the limit's wall-clock end, and past it no more polynomials are root-optimized; a first round ends near its limit (59 → 67 s, 166 → 176 s, 333 → 345 s). When no polynomial reaches msieve's minimum Murphy E (the c85), selection searches again with a quarter of it in half the time, up to three times; a retry ends 60–80 s past its limit.

The time limits themselves, set for CPU selection, were then most of a run. Shorter limits from scratch: c95 333 s (msieve's) 413 s total, 167 s 248 s, 83 s 163 s, 42 s 124 s, all sieving in 43.0–43.6 s; c90 166 s 395 s, 83 s 294 s, 42 s 254 s, 21 s 236 s, each with a retry of about 140 s. So polynomials are now kept down to a quarter of the minimum E (`MSIEVE_POLY_KEEP_E`; the Bernstein bound stays msieve's, so no more Murphy E work), and the best one found is used when none reaches the minimum; and GPU builds take a quarter of msieve's limits (`MSIEVE_POLY_TIME_SCALE`; 1/32 since Nov 10, below). From scratch, `./msieve` without options:

Digits	MPQS	NFS, msieve's limits	**NFS, a quarter**	An eighth
85	350 s	227 s	**93.4 s**	—
90	981 s	395 s	**110.6 s**	89.6 s
95	2,800 s	413 s	**163.4 s**	124.2 s

No retries, the same or better polynomials (c95: E 2.745e-08, sieving 43.0 s, as with the full limit), all factors correct. An eighth found the same polynomials, but on two numbers; the default is a quarter.

Larger inputs, from scratch, a quarter against msieve's full limits: c100 291 s against 808 s (the same polynomial, sieving 71.2 s); c105 516 s against 1,386 s (E 6.339e-09 against 6.649e-09, sieving 116.7 s against 118.4 s); c110, degree 5, 1,229 s with a quarter (sieving 283.4 s, matrix 297,952 columns). At 110 digits the first round ended after 264 s (degree 5's stage 1 counts four GPU threads' time) with nothing past the Bernstein bound, and the retry found the polynomial; it also kept 485,371 polynomials above a quarter of the minimum and wrote each, so those below the minimum are now only kept when they are the best so far.

Three more fixes, from that c110: stage 1 compared its four GPU threads' time added up with the limit (now their average); degree 5's Bernstein bound (1/6.6 of the minimum E at 110 digits, against degree 4's 1/30000) let nothing through in a whole first round, so it is now derived from the lower minimum that is kept; and at the limit the GPU root optimization finished every job it had started (306 s of them for degree 5), so near the limit it now starts a job only if it can end in time (each thread's average job time) and drops the queued ones past it. Reporting uses msieve's minimum E across retries. From scratch: c85 93.4 → 85.9 s, c95 163.4 → 152.6 s, c110 1,229 → 1,022.7 s (selection 687 s for a 675 s limit, no retry), the same polynomials.

Below 85 digits: msieve's polynomial parameters start at 80 digits and an 80-digit input (79.99 there) found none; up to 5 digits below the table now use its first entry. NFS against MPQS from scratch: c80 72.8 against 132.4 s, c81 87.8 against 186.5 s, c82 76.8 against 265.8 s, c83 83.4 against 267.9 s, c84 82.3 against 382.4 s. The NFS wins throughout, so the default now starts at 80 digits, as low as msieve's NFS goes (`MIN_NFS_BITS`). Below that, with a test build whose NFS minimum is lowered (`-DMIN_NFS_BITS=230`): c75 MPQS 49.1 s, the NFS stops (no polynomial parameters); c76 54.2 s against an NFS whose matrix had more rows than columns (msieve's parameters gather too few relations there); c77 69.4 against 78.8 s; c78 98.2 against 80.6 s; c79 109.3 against 82.4 s. The crossover is near 78 digits, but the gain there is 17–27 s, so the default stays at 80 digits and msieve's minimum is unchanged.

Above 110 digits (degree 5, from scratch, a quarter of the limits): c115 28.5 min (selection 20.6 min, sieving 403.3 s, matrix 370,886 columns, peak 6.6 GB), with msieve's full limit 90.0 min and the same polynomial (E 6.432e-10); c120 52.7 min (selection 40.1 min, sieving 654.8 s, matrix 517,766 columns, peak 7.2 GB). All factored correctly in one sieving round. Polynomial selection is now 76 % of a c120 run: its limit doubles every 5 digits, sieving grows about 1.6 times.

CADO-NFS on the same c120 (CPU only, 32 threads, default parameters): polynomial selection 2.6 min (this fork 40.1), sieving 19.8 min, 16,770 CPU-seconds, 16.2 M relations (10.9 min, 655 s, 12.1 M), filtering 0.8 min (0.5), linear algebra 4.6 min, 5,305 CPU-seconds, matrix 635,213 rows (0.6 min, 517,766 columns), characters and square root 0.6 min (0.2); total 28.6 min (52.7). One RTX PRO 4500 did the sieving of 25.6 CPU threads of CADO-NFS's `las` and the linear algebra of about 150 threads of its `bwc`; msieve's polynomial selection loses the run.

10.4 RSA-2048: an extrapolated estimate

An extrapolation, an order of magnitude at best. RSA-2048 (617 digits) cannot be factored with this fork, CADO-NFS or any existing hardware; the numbers say how far out of reach it is. The number field sieve's heuristic cost $L(N) = \exp((64/9)^{1/3}(\ln N)^{1/3}(\ln \ln N)^{2/3})$ (the $o(1)$ term ignored) is scaled from two references: RSA-250 (829 bits), factored by the CADO-NFS team in 2020 in about 2,700 CPU-core-years (Xeon Gold 6130; taken as 2,450 sieving and 250 linear algebra) *without GPUs*, with a matrix of about 405 million rows; and the c120 above, which converts CPU to GPU work (sieving 25.6 threads per GPU, linear algebra about 150), an i9-13900K thread taken as one Xeon core (uncertain within a factor of two or so).

Quantity	Estimate
L(RSA-2048)/L(RSA-250)	2.3×10^{11}
L(RSA-2048)/L(c120)	1.0×10^{18}
RSA-250 on RTX PRO 4500-class GPUs	about 97 GPU-years
RSA-2048, CPU	about 6×10^{14} core-years
RSA-2048, GPUs, from RSA-250 / from this fork's c120	2.3×10^{13} GPU-years (both)
with 10^4 / 10^6 / 10^8 GPUs, perfectly parallel	2×10^9 / 2×10^7 / 2×10^5 years
matrix dimension ($\sqrt{L}$, from RSA-250 / from the c120)	2×10^{14} to 6.5×10^{14}
matrix memory (110 entries per row, 4 bytes)	0.09 to 0.3 exabytes
96 GB GPUs to hold the matrix	about 10^6
relations ($\sqrt{L}$, from RSA-250 / the c120)	4×10^{15} to 1.7×10^{16}

The two time estimates agree, which says only that the c120 conversion is consistent with the record: the $\sqrt{L}$ rule for the matrix already misses the c120 by a factor of 3.3 (191,000 rows predicted from RSA-250, 635,213 measured). The conclusion does not depend on the error: RSA-2048 costs about 10^{11} times RSA-250; with a hundred million GPUs it would take some 10^5 years, and its matrix alone would need about a million 96 GB GPUs.

The square root, several dependencies at once. msieve's `nfs_find_factors` now tries `MSIEVE_SQRT_THREADS` dependencies (default 4) at a time: their relations are read in one pass (`nfs_read_dependencies`: the union read once from the store or the savefile, a copy per dependency), each runs on its own thread with its own copies of the monic polynomial and of `msieve_obj` (the random seeds), and the results are taken in dependency order with the serial loop's stopping rule. The derivative factor, the same for every dependency, is computed once (`eval_poly_derivative` changes its argument while it runs). On the c95 (`-nc3`, three sets of dependencies): when dependency 1 gives the factors 6.2 s one at a time against 6.5 s four at a time; when it does not, 12.2 s against 6.5 s. Four take as long as one and need a second round only when all four fail (1 in 16). `./msieve -n`: 68.0 s, square root 4 s.

Each step's gain, q 2,200,000–2,201,000 (72 lattices), the siever's log line without startup; the first two rows were measured with the line siever's run on the same machine:

Version	Relations	Total	GPU	Host	Batch	To the host
first: the host sorts the resieved hits and joins them to the candidates	25,468	4.3 s	1.7 s	2.4 s	0.2 s	2,235,499
hits into each candidate's list through a GPU hash table, rational test on the GPU	25,468	1.4 s	0.3 s	0.1 s	0.2 s	815,082
one lattice per `la_large` launch, primes below 32 not sieved, idle machine	25,658	0.7–0.8 s	0.1 s	0.1 s	0.2 s	1,023,712
the three fixes	**25,768**	**0.7 s**	0.1 s	0.1–0.2 s	0.2 s	1,029,718

The first full run (18,672 lattices, sharing the machine with the line siever's run) took 838 s, 668 s of it host factoring.

Kernel time per 72 lattices (nsys, idle GPU, the fixed version), by the bound below which primes are not sieved:

Not sieved	`la_tile`	`la_large`	roots, rows, filter	Candidates	Relations
none	147.3 ms	74.4 ms	3.4 ms	2,548,519	26,574
below 16	62.3 ms	76.2 ms	3.7 ms	3,193,573	26,448
below 32 (default)	53.6 ms	76.5 ms	3.6 ms	3,129,055	25,768
below 64	48.2 ms	75.8 ms	3.6 ms	3,079,208	25,001

Sieving every prime finds 3 % more relations than the default but costs 94 ms more `la_tile` time per 72 lattices, about 24 s more GPU time for the c95; a bound of 16 finds 2.6 % more for 9 ms. Before the fixes the default found 25,658 relations with `la_tile` at 51.9 ms. With all lattices of a batch in one `la_large` launch, `la_large` (both passes) took 416.6 ms (measured with the line siever on the GPU too), against 113.8 ms with one lattice per launch under the same conditions: the batch's sieve arrays did not fit in L2, so the atomic adds went to DRAM.

11 The whole run, end to end (November 10)

With every stage on the GPU or on all CPUs, a 120-digit number from scratch took 52.7 min, and CADO-NFS on 32 CPU threads 28.6 min: msieve's polynomial selection, 40.1 min of it, lost the run. This section is the work that followed, each step judged on whole factorizations from scratch (`./msieve` with its defaults, factors checked) and, for the kernels, on a fixed c120 range (special-q 4,961,532–5,100,000, 8,946 lattices) whose relations were compared with a baseline after every change.

c120 from scratch	Selection	Sieving	Total
Nov 9: a quarter of msieve's polynomial limits	40.1 min	654.8 s	52.7 min
1/32 of the limits, root optimization of the 48 best by size	5.1 min	682.3 s	18.2 min
lattice siever in 32-bit arithmetic	5.1 min	516.3 s	15.4 min
one sieve array kept in L2	5.1 min	429.2 s	13.95 min
1/128 of the limits	1.4 min	429.3 s	10.2 min
`la_tile`: vector loads, no bank conflicts, 8,192-position tiles	1.4 min	380.2 s	9.8 min

Afterwards the square root tries 8 dependencies at once and does its large products on the GPU, and the free relations are found on all CPUs (about 5 s more off the c120).

11.1 Polynomial selection: shorter time limits

Scored with CADO-NFS's Murphy E (its c120 bounds and area), the c120's polynomial was 3.691e-06 at 1/32 of msieve's limit (6.4 min), the same at 1/16, 3.751e-06 at 1/8 (20.1 min) and at 1/4 (40.1 min); CADO-NFS's 2.6-minute selection 3.794e-06. `MSIEVE_ROOTOPT_TOPK=`K root-optimizes a size-optimized polynomial only if its size score is among the K best so far, as CADO-NFS's `nrkeep`; but among the survivors of the stage 2 bound the size score hardly predicts the final E (Spearman -0.09 over 7,917 c120 polynomials, measured with an `R` line per root optimization in `MSIEVE_SIZEOPT_LOG`), so it is a random subset in effect: with $K = 48$ the freed GPU gave 2.4 times the survivors and no better polynomial. It does end a short selection at its limit. GPU builds now take 1/32 of the limits and $K = 48$. From scratch (total, a quarter → now; all factored correctly):

Input	A quarter	**Now**	Selection	Sieving (before → now)
c85	85.9 s	**76 s**	4 s	34–43 → 43.6 s
c95	152.6 s	**84 s**	12 s	43.0 → 46.1 s
c100	291 s	**129 s**	25 s	71.2 → 74.2 s
c105	516 s	**195 s**	43 s	116.7 → 114.9 s
c110	1,023 s	**437 s**	90 s	283.4 → 283.4 s
c115	28.5 min	**11.1 min**	161 s	403.3 → 432.4 s
c120	52.7 min	**18.2 min**	5.1 min	654.8 → 682.3 s

The weaker polynomials cost 0–7 % more sieving. The c120 now takes 18.2 min, against CADO-NFS's 28.6 min on 32 CPU threads.

11.2 The lattice siever: 32-bit arithmetic, a sieve array kept in L2

The kernels used 64-bit integers throughout, and the GPU emulates 64-bit division and modulo. In 32 bits where the values fit (`la_large`'s reduction and walk for $p < 2^{30}$, with the j steps clamped to J; `la_small`'s $(rj+I/2) \bmod p$; `la_roots`'s inverse): on 8,946 c120 lattices `la_large` 9,752 → 8,028 ms, `la_tile` 4,706 → 2,653 ms, GPU time 15.9 → 12.0 s, relations identical, both check modes clean. Full c120: sieving 682.3 → 516.3 s, total 18.2 → 15.4 min. `la_large`'s atomics run at about 17 G/s (10.38 M hits per lattice in 0.69 ms) against 38 G/s for random atomics into 32 MB in a microbenchmark; reusing two L2-resident sieve arrays and other grid sizes did not help. Nsight Compute: L2 22 %, DRAM 27 %, issue 6 % busy, 45 cycles of long-scoreboard stall per instruction, each warp-wide atomic carrying 18.6 of 32 lanes. `red.global` instead of a generic `atom.add`, and four adds in flight per thread, gave identical relations and no gain, as did a per-warp queue making every atomic add 32 lanes wide. The counters also showed 10 % of the atomic sectors missing L2: a batch had a 32 MB sieve array per lattice, each sieved by its own launch and read later, so every launch started cold. With one array (lattice k sieved by `la_large`, read and cleared by `la_tile`, then reused for $k+1$ while in L2; two alternating arrays fill the whole 64 MB L2 and gained nothing): `la_large` 8,028 → 6,367 ms, GPU time 12.0 → 9.9 s, relations identical. Full c120: sieving 516.3 → 429.2 s, total 15.4 → 13.95 min.

11.3 1/128 of the time limits

With sieving faster, the c120 found the same polynomial at 1/64 and 1/128 of msieve's limits as at 1/32 (82 s instead of 5.1 min). From scratch at 1/128: c85 68 s, c95 66 s, c100 90 s, c105 128 s, c110 302 s, c115 388 s, c120 610 s (10.2 min), all factored correctly; the polynomials were the same as at 1/32 except c85 and c95 (slightly better) and c110 (3.6 % lower E, 335 s at 1/32 against 302 s). GPU builds now default to 1/128.

11.4 `la_tile`: vector loads, no bank conflicts, smaller tiles

Nsight Compute: 33 % occupancy (a 64 KB tile, one block an SM), a fifth of the stall samples on loading the tile a word at a time, 18 M shared bank conflicts a lattice in the candidate scan (every lane of a warp on the same bank). The tile now moves 16 bytes at a time and each lane starts its run at its own offset (relations identical, `la_tile` 2,611 → 1,985 ms on the test range); tiles of 8,192 positions give three blocks an SM (1,465 ms). Each thread's thresholds then come from a run of 16 positions instead of 32: 328,760 relations instead of 328,657, all valid. Full c120: sieving 429.3 → 380.2 s, total 9:49.

11.5 The square root: more dependencies, products on the GPU

With 32 CPUs or more, 8 dependencies are tried at once (about the same average time as 4, without the second round that 4 needs one time in 16: the c120 run that hit it went 64 → 34 s). Its products whose operands both have at least 2,048 limbs go to the GPU (`gnfs/sqrt/sqrtmul_core.cu`: an NTT modulo three primes $c \cdot 2^{40} + 1$, CRT, carries on the host; the same integer as GMP's; ported from the author's GPU square root for CADO-NFS, driver API and PTX with an interface check). All 38,688 products of a c120 square root matched GMP's in the check mode. The c120's square root: 33.7 → 29.1 s; reading the relations (8 s) and the many small polynomial products and reductions stay on the CPU.

11.6 Free relations

Free relations are now tested on all CPUs (6 s of the c120's filtering before; the same relations, the same matrix).

11.7 Larger inputs: c130 and c140

From scratch with all of the above: c130 35.4 min (selection 5.2 min, sieving 1,515 s, filtering 61 s, linear algebra 144 s, square root 85 s), c140 109.6 min (selection 19.5 min, sieving 4,437 s, filtering 148 s, linear algebra 578 s on a 2.96 M matrix, square root 233 s, peak 13.2 GB), both factored correctly. The CADO-NFS GPU fork does the same c140 in 75.9 min on this machine. msieve's c140 polynomial scores 1.3 % below CADO-NFS's on CADO-NFS's Murphy E. The GPU square root saves 17 % at both sizes.

Stage	c120	c130	c140
Polynomial selection (1/128 of msieve's limits)	1.4 min	5.2 min	19.5 min
Sieving	380 s	1,515 s	4,437 s
Filtering	about 31 s	61 s	148 s
Linear algebra (matrix columns)	34 s (517 k)	144 s (1.32 M)	578 s (2.96 M)
Square root	about 29 s	85 s	233 s
Total	**9.8 min**	**35.4 min**	**109.6 min**
Peak memory	7.2 GB	7.5 GB	13.2 GB

At 140 digits the CADO-NFS GPU fork is the faster program on this machine (75.9 min: sieving 58 min against 74, linear algebra 3.7 min against 9.6), and polynomial selection takes 19.5 min at 1/128 of msieve's limits, which grow about four times every 10 digits; whether less would do there was not measured.

12 Build, settings and the C11 code

The GPU build,

`PATH=/usr/local/cuda-13.2/bin:$PATH make all CUDA=13`
(`GPU_LATTICE=0`: the lattice choice on the CPU),

writes `stage1_core_sm{80,86,89,120}.ptx` and `sizeopt_core_sm*.ptx` (the latter with `-fmad=false` and `-I$(CGBN_INCLUDE)`); msieve loads them from the current directory. Settings: `MSIEVE_STAGE2_THREADS` (online CPUs), `MSIEVE_GPU_SIZEOPT` (on), `MSIEVE_Q_THREADS` (4), `MSIEVE_CPU_LIFT`, `MSIEVE_OLD_CRT` (old paths for comparison), `MSIEVE_SIZEOPT_LOG` (measurement log), and `-DQ_ROOTS_MIN` at compile time. Root optimization runs on the GPU by default (§7; `MSIEVE_GPU_ROOTOPT=0` for msieve's code), with `MSIEVE_ROOTOPT_THREADS` and `_SLOTS`, `MSIEVE_ROOTOPT_BATCHED`, `MSIEVE_GPU_LINESIEVE`, `MSIEVE_GPU_RSSETUP`, `MSIEVE_GPU_LINECAND`, `MSIEVE_GPU_ALPHA`, `MSIEVE_GPU_BERNSTEIN`, `MSIEVE_GPU_MURPHY`, `MSIEVE_ROOTOPT_CPU_SHARE` and `MSIEVE_ROOTOPT_STATS`. The modules are also built for sm_90 and sm_100 (H100, B200), which used to load the sm_80 code. A `CUDA=13` build writes, for each architecture, `stage1_core`, `sizeopt_core`, `lanczos_core`, `sieve_core`, `lasieve_core` and `sqrtmul_core` PTX files, all loaded from the current directory; the lattice siever and the square root check that theirs match the binary (`la_abi`, `sm_abi`) and fall back to the line siever or GMP otherwise. Every build option, compile-time macro and environment variable is described in the companion document *msieve GPU fork: build options and runtime knobs* (`msieve-gpu-knobs.pdf`).

msieve was already ANSI C with prototypes. The tree now builds with `-std=c11 -D_GNU_SOURCE -D_DEFAULT_SOURCE` and `-Wall -W -Wpedantic -Wstrict-prototypes -Wold-style-definition -Wmissing-prototypes` with zero warnings in the CUDA, CPU-only and ECM builds (`-Wno-overlength-strings` for one 5024-byte string in `optimize_deg6.c`). The changes: `asm` as `__asm__`, `__extension__` on the 128-bit type, `f(void)`, file-local functions made static, empty translation units given a typedef, unused parameters cast to void, sign-compare casts that keep the old conversions, intended fall-throughs marked, and the deprecated CUDA device queries replaced by `cuDeviceGetAttribute`. Two warnings were real bugs: the CPU stage 1 used a pointer after `realloc` had moved its block (it now keeps the offset), and the demo left an over-long number argument unterminated. Stage 1 hits, size- and root-optimization outputs and MPQS/ECM factorizations were compared with the original build (with `-ffp-contract=off` for the floating-point outputs).

13 Open points

- Line sieving: over a whole run the GPU is the limit (§9); several GPUs in one process.
- Polynomial selection: whether less than 1/128 of the limits would do at 140 digits and above (19.5 min of the c140's 110), and a better predictor than the size score of what is worth root-optimizing.
- Lattice sieving at 140 digits: CADO-NFS's GPU siever is faster (58 against 74 min); `la_large`'s atomic adds run at about half the rate of a microbenchmark, for reasons the counters did not settle.
- The production GPUs: nothing was measured on an RTX PRO 6000 or a B200.
- Lattice sieving: clique removal and the merge (about 1 s each) are sequential; a bound of 16 for the primes not sieved, untested on whole runs; $I = 4096$ and msieve's line-siever parameters throughout (whole runs c85–c140).
- `read_poly` (`gnfs/poly/poly.c`) reports success when the `.fb` file is for a different number; worth deciding separately.
- Root optimization on a data-centre GPU: Murphy E on the GPU and the CPU-share policy are untested on such hardware. GMP left in root optimization: the size optimization of `-npr` input, the CPU fallbacks and the output of saved polynomials.
- A speedup on several real GPUs is untested (one card here; the multi-GPU paths were run with `MSIEVE_SIMULATE_GPUS`).

- Degree 6 size optimization (`optimize_deg6.c`) is not on the GPU.
- GPU performance counters (`ncu`) need administrator rights on the test machine; the lattice siever's kernels were profiled twice that way (scripts in `runs/prof120/`), the rest with nsys and counters compiled into a scratch copy.